

✓ R : Assignment - 3

Aditya Pradeep Singh

CMPN - B

23102B0010

```
install.packages(c("dplyr","jsonlite","readxl","writexl","RSQLite","DBI"), quiet = TRUE)

library(dplyr)
library(jsonlite)
library(readxl)
library(writexl)
library(RSQLite)
library(DBI)
```

Organizing the dataset into the following files:

- transactions.csv – InvoiceNo, StockCode, CustomerID, Quantity, InvoiceDate
- products.json – StockCode, Description, UnitPrice
- customers.xlsx – CustomerID, Country

```
raw_path <- "Online Retail.xlsx"

transactions_path <- "transactions.csv"
products_path <- "products.json"
customers_path <- "customers.xlsx"

if (file.exists(raw_path)) {

  message("Found Online Retail.xlsx – splitting into transactions/products/customers...")

  raw <- read_excel(raw_path)

  transactions <- raw %>%
    select(InvoiceNo, StockCode, CustomerID, Quantity, InvoiceDate)
  write_csv(transactions, transactions_path, row.names = FALSE)

  products <- raw %>%
    filter(!is.na(StockCode)) %>%
    group_by(StockCode) %>%
    slice(1) %>%
    ungroup() %>%
    select(StockCode, Description, UnitPrice)
  write_json(products, products_path, auto_unbox = TRUE)

  customers <- raw %>%
    filter(!is.na(CustomerID)) %>%
    distinct(CustomerID, Country)
  writexl::write_xlsx(customers, customers_path)

  message("Split complete: transactions.csv, products.json, customers.xlsx created.")

} else if (file.exists(transactions_path) && file.exists(products_path) && file.exists(customers_path)) {

  transactions <- read_csv(transactions_path, stringsAsFactors = FALSE)
  products <- fromJSON(products_path)
  customers <- read_excel(customers_path)

} else {

  message("Real data files not found – generating sample data instead.")

  set.seed(42)
  n <- 500

  transactions <- data.frame(
    InvoiceNo = sample(10000:10999, n, replace = TRUE),
    StockCode = sample(paste0("P", 1:20), n, replace = TRUE),
    CustomerID = sample(c(NA, 1:50), n, replace = TRUE, prob = c(0.03, rep(0.97/50, 50))),
    Quantity = sample(c(-2, 0, 1:10), n, replace = TRUE),
    InvoiceDate = sample(seq(as.Date("2023-01-01"), as.Date("2023-12-31"), by = "day"), n, replace = TRUE),
    stringsAsFactors = FALSE
  )
  transactions$Quantity[sample(1:n, 5)] <- NA
```

```

products <- data.frame(
  StockCode = paste0("P", 1:20),
  Description = paste("Product", 1:20),
  UnitPrice = round(runif(20, -1, 50), 2)
)
products$UnitPrice[products$UnitPrice < 0] <- 0

customers <- data.frame(
  CustomerID = 1:50,
  Country = sample(c("UK", "Germany", "France", "USA", "India", "Australia"), 50, replace = TRUE)
)

```

Found Online Retail.xlsx – splitting into transactions/products/customers...

Split complete: transactions.csv, products.json, customers.xlsx created.

Task 1: Import and Clean the Data

```

cat("---- Raw dataset dimensions ----\n")
cat("Transactions:", dim(transactions), "\n")
cat("Products:", dim(products), "\n")
cat("Customers:", dim(customers), "\n")

transactions_clean <- transactions %>%
  filter(!is.na(CustomerID), !is.na(Quantity), !is.na(StockCode)) %>%
  distinct() %>%
  filter(Quantity > 0)

products_clean <- products %>%
  distinct() %>%
  filter(!is.na(UnitPrice), UnitPrice > 0)

customers_clean <- customers %>%
  distinct() %>%
  filter(!is.na(CustomerID))

cat("\n---- Cleaned dataset dimensions ----\n")
cat("Transactions:", dim(transactions_clean), "\n")
cat("Products:", dim(products_clean), "\n")
cat("Customers:", dim(customers_clean), "\n")

```

```

---- Raw dataset dimensions ----
Transactions: 541909 5
Products: 4070 3
Customers: 4380 2

```

```

---- Cleaned dataset dimensions ----
Transactions: 392708 5
Products: 3855 3
Customers: 4380 2

```

Task 2: Integrate the Multiple Data Sources

```

integrated_data <- transactions_clean %>%
  inner_join(products_clean, by = "StockCode") %>%
  left_join(customers_clean, by = "CustomerID") %>%
  mutate(Revenue = Quantity * UnitPrice)

cat("\n---- Integrated dataset dimensions ----\n")
print(dim(integrated_data))

unmatched_customers <- sum(is.na(integrated_data$Country))
cat("Transactions with no matching customer country:", unmatched_customers, "\n")

```

```

Warning message in left_join(., customers_clean, by = "CustomerID"):
"Detected an unexpected many-to-many relationship between `x` and `y`.
i Row 196 of `x` matches multiple rows in `y`.
i Row 1 of `y` matches multiple rows in `x`.
i If a many-to-many relationship is expected, set `relationship =
  "many-to-many"` to silence this warning."

```

```

---- Integrated dataset dimensions ----

```

```
[1] 388774      9
Transactions with no matching customer country: 0
```

Task 3: Perform Sales and Customer Analysis

```
total_revenue <- sum(integrated_data$Revenue, na.rm = TRUE)
cat("\nTotal Sales Revenue:", total_revenue, "\n")

top5_products <- integrated_data %>%
  group_by(StockCode, Description) %>%
  summarise(TotalRevenue = sum(Revenue), .groups = "drop") %>%
  arrange(desc(TotalRevenue)) %>%
  head(5)
cat("\nTop 5 Products by Revenue:\n")
print(top5_products)

top5_countries <- integrated_data %>%
  filter(!is.na(Country)) %>%
  group_by(Country) %>%
  summarise(TotalRevenue = sum(Revenue), .groups = "drop") %>%
  arrange(desc(TotalRevenue)) %>%
  head(5)
cat("\nTop 5 Countries by Revenue:\n")
print(top5_countries)

top5_customers <- integrated_data %>%
  group_by(CustomerID) %>%
  summarise(TotalRevenue = sum(Revenue), .groups = "drop") %>%
  arrange(desc(TotalRevenue)) %>%
  head(5)
cat("\nTop 5 Customers by Revenue:\n")
print(top5_customers)

customer_value <- integrated_data %>%
  group_by(CustomerID) %>%
  summarise(TotalRevenue = sum(Revenue), .groups = "drop") %>%
  mutate(ValueSegment = case_when(
    TotalRevenue < 100 ~ "Low Value",
    TotalRevenue < 500 ~ "Medium Value",
    TotalRevenue < 1000 ~ "High Value",
    TRUE ~ "Premium"
  ))

cat("\nCustomer Value Segmentation (sample):\n")
print(head(customer_value, 10))

cat("\nSegment counts:\n")
print(table(customer_value$ValueSegment))

best_market <- top5_countries$Country[1]
country_revenue_all <- integrated_data %>%
  filter(!is.na(Country)) %>%
  group_by(Country) %>%
  summarise(TotalRevenue = sum(Revenue), .groups = "drop") %>%
  arrange(TotalRevenue)
worst_market <- country_revenue_all$Country[1]

cat("\nHigh-performing market:", best_market,
    "\n- highest total revenue, indicating strong demand/customer base.\n")
cat("\nUnderperforming market:", worst_market,
    "\n- lowest total revenue, suggesting limited reach or lower purchasing activity.\n")
```

Total Sales Revenue: 10781592

Top 5 Products by Revenue:

```
# A tibble: 5 × 3
  StockCode Description TotalRevenue
<chr>      <chr>          <dbl>
1 23843    PAPER CRAFT , LITTLE BIRDIE 168470.
2 47566    PARTY BUNTING 142633.
3 22423    REGENCY CAKESTAND 3 TIER 135813.
4 85123A   WHITE HANGING HEART T-LIGHT HOLDER 94067.
5 23166    MEDIUM CERAMIC TOP STORAGE JAR 81033.
```

Top 5 Countries by Revenue:

```
# A tibble: 5 × 2
  Country TotalRevenue
<chr>      <dbl>
1 United Kingdom 8861857.
```

```

2 Netherlands      363884.
3 EIRE              331660.
4 Germany           263819.
5 France            226976.

```

Top 5 Customers by Revenue:

A tibble: 5 × 2

```

  CustomerID TotalRevenue
    <dbl>         <dbl>
1     18102     408760.
2     14646     357531.
3     17450     186038.
4     14911     182691.
5     16446     168472.

```

Customer Value Segmentation (sample):

A tibble: 10 × 3

```

  CustomerID TotalRevenue ValueSegment
    <dbl>         <dbl> <chr>
1     12346      77184. Premium
2     12347      5438. Premium
3     12348      1790. Premium
4     12349      1927. Premium
5     12350       408. Medium Value
6     12352      1617. Premium
7     12353       171. Medium Value
8     12354      1241. Premium
9     12355       546. High Value
10    12356      3709. Premium

```

Segment counts:

High Value	Low Value	Medium Value	Premium
931	117	1399	1892

High-performing market: United Kingdom - highest total revenue, indicating strong demand/customer base.

Underperforming market: Saudi Arabia - lowest total revenue, suggesting limited reach or lower purchasing activity.

Task 4: Store and Retrieve Data Using SQL

```

con <- dbConnect(RSQLite::SQLite(), "retail_sales.db")
dbWriteTable(con, "retail_sales", integrated_data, overwrite = TRUE)

```

```

cat("\n---- SQL Query 1: Top 5 customers by revenue ----\n")

```

```

query1 <- dbGetQuery(con, "
  SELECT CustomerID, SUM(Revenue) AS TotalRevenue
  FROM retail_sales
  GROUP BY CustomerID
  ORDER BY TotalRevenue DESC
  LIMIT 5
")
print(query1)

```

```

cat("\n---- SQL Query 2: Total revenue by country ----\n")

```

```

query2 <- dbGetQuery(con, "
  SELECT Country, SUM(Revenue) AS TotalRevenue
  FROM retail_sales
  WHERE Country IS NOT NULL
  GROUP BY Country
  ORDER BY TotalRevenue DESC
")
print(query2)

```

```

dbDisconnect(con)

```

```

---- SQL Query 1: Top 5 customers by revenue ----

```

```

  CustomerID TotalRevenue
1      18102     408760.0
2      14646     357531.1
3      17450     186038.0
4      14911     182690.5
5      16446     168472.5

```

```

---- SQL Query 2: Total revenue by country ----

```

```

  Country TotalRevenue
1 United Kingdom  8861857.13
2 Netherlands    363884.48
3 EIRE           331660.17
4 Germany        263818.97
5 France         226975.60
6 Australia      173918.61
7 Spain          74983.96

```

8	Switzerland	67567.19
9	Belgium	55134.05
10	Japan	48600.22
11	Sweden	43652.09
12	Norway	40283.20
13	Portugal	32679.45
14	Finland	23306.79
15	Denmark	22635.95
16	Channel Islands	22059.72
17	Italy	21065.45
18	Cyprus	19219.77
19	Austria	19119.46
20	Singapore	11497.98
21	Poland	10264.54
22	Israel	9088.79
23	Greece	6128.80
24	Iceland	5438.44
25	Canada	4315.90
26	USA	4138.83
27	Unspecified	2952.93
28	Malta	2890.34
29	United Arab Emirates	2747.72
30	Lebanon	2085.13
31	Lithuania	1804.34
32	European Community	1575.65
33	Brazil	1225.35
34	RSA	1090.43
35	Czech Republic	1026.26
36	Bahrain	716.78
37	Saudi Arabia	181.44

```
cat("\n---- Business Insights ----\n")
cat("1.", best_market, "generates the highest revenue and should be prioritized for marketing spend and stock allocation.\n")
cat("2. A small set of top products/customers contribute disproportionately to revenue, supporting a focused retention strat\n")
cat("3.", worst_market, "underperforms relative to other markets and may need targeted promotions or a review of pricing/log\n")
```

```
---- Business Insights ----
1. United Kingdom generates the highest revenue and should be prioritized for marketing spend and stock allocation.
2. A small set of top products/customers contribute disproportionately to revenue, supporting a focused retention strategy f
3. Saudi Arabia underperforms relative to other markets and may need targeted promotions or a review of pricing/logistics ir
```